Listing 36.2 Unit testing `StatusMiddleware` when a nonmatching `path` is provided

```
[Fact]
public async Task ForNonMatchingRequest_CallsNextDelegate()
{
    var context = new DefaultHttpContext();                    ❶
    context.Request.Path = "/somethingelse";                   ❶
    var wasExecuted = false;                                   ❷
    RequestDelegate next = (HttpContext ctx) =>                ❸
    {                                                          ❸
        wasExecuted = true;                                    ❸
        return Task.CompletedTask;                             ❸
    };                                                         ❸
    var middleware = new StatusMiddleware(next);               ❹

    await middleware.Invoke(context);                          ❺

    Assert.True(wasExecuted);                                  ❻
}
```

❶ Creates a DefaultHttpContext and sets the path for the request

❷ Tracks whether the RequestDelegate was executed

❸ The RequestDelegate representing the next middleware should be invoked in this example.

❹ Creates an instance of the middleware, passing in the next RequestDelegate

❺ Invokes the middleware with the HttpContext; should invoke the RequestDelegate

❻ Verifies that RequestDelegate was invoked

When the middleware is invoked, it checks the provided `Path` and finds that it doesn't match the required value of `/ping`. The middleware therefore calls the next `RequestDelegate` and returns.

The other obvious case to test is when the request `Path` is `"/ping"`; the middleware should generate an appropriate response. You could test several characteristics of the response:

- The response should have a `200 OK` status code.